GUÍA PRÁCTICA · NEIRO AGENCY

Mods de Claude Code: qué son, cómo instalarlos y para qué sirven



Un mod es un plugin de Claude Code hecho de funciones en JavaScript o TypeScript que corren dentro del programa: pueden dibujar paneles y barras, reaccionar a lo que Claude hace en cada sesión y hasta cambiarlo. Esta guía resume lo que dice la documentación oficial: cómo se instalan, cómo se crean, qué usos tienen y qué cuidados hay que tener.

Información al 1 de noviembre de 2026. Los mods son nuevos y la documentación avisa que sus eventos y métodos pueden cambiar entre versiones, así que lo que se describe aquí puede variar con el tiempo.

Marcos Vargas IA · Sistemas · Automatización

LO QUE APRENDERÁS

01	Qué es un mod y en qué se diferencia de hooks, skills y MCP
02	Dónde se ven y en qué versiones funcionan
03	Cómo instalarlos y probar los ejemplos de Anthropic
04	Cómo crear uno, probarlo y compartirlo
05	Usos concretos que muestra la documentación
06	Seguridad y control para organizaciones
07	Cómo lo uso yo

SECCIÓN 01
QUÉ ES

Funciones que corren dentro de Claude Code y reaccionan a eventos.

Según la documentación oficial, un mod es un plugin que cambia cómo se ve y se comporta Claude Code. Claude Code llama a sus funciones (“hooks”) cuando ocurre un evento: una llamada a una herramienta, un prompt enviado o una parte de la interfaz que se va a dibujar. Cada función puede **observar** el evento y dejarlo pasar, **reescribirlo** antes de que siga, o **responderlo** ella misma (por ejemplo, rechazar un comando).

Como corre por dentro, un mod puede dibujar un panel al lado de la conversación o una banda encima del prompt (con pestañas, botones y campos de texto), rediseñar partes que Claude Code ya dibuja (el spinner o la fila de una herramienta), ejecutar un comando `/lo-que-sea` propio al instante, y compartir datos entre sus funciones. Una función puede contar llamadas y otra mostrar el conteo junto al spinner.

	MOD	HOOK DE SETTINGS	SKILL	SERVIDOR MCP
Qué es	Funciones dentro de un plugin, en el proceso de Claude Code	Comando, HTTP o prompt que se ejecuta en un evento	Un `SKILL.md` con instrucciones	Proceso externo que da herramientas
¿Dibuja en la interfaz?	Sí	No	No	No
Elegilo cuando…	Querés un panel, una banda, un comando propio o reescribir un evento	Querés bloquear, permitir o registrar con un script que ya tenés	Repetís las mismas instrucciones	Claude necesita llegar a un sistema externo

SECCIÓN 02
DÓNDE SE VEN

Los hooks corren casi en todas partes; el dibujo, no.

DÓNDE USÁS CLAUDE CODE	QUÉ PASA CON UN MOD
Terminal (`claude`)	Los hooks corren y lo que dibuja se ve. Pide Claude Code 2.1.287 o más nuevo.
Pestaña Code de la app de escritorio	Corren y se ve, salvo los elementos marcados solo-terminal. Funciona desde la 2.1.286. En una sesión WSL no hay plugins.
Extensión de VS Code	Corren y se ve con la extensión 2.1.299 o más nueva (2.1.304 si la sesión no obtiene feature flags, por ejemplo en Bedrock).
`claude -p` y Agent SDK	Los hooks corren, pero no se dibuja nada.
Sesión en la nube	Los hooks corren si el plugin llega a la sesión; no se dibuja.

Los mods vienen activados por defecto. Para comprobar tu versión, corré `claude --version`; para ver los mods cargados en una sesión, escribí `/plugin` (una línea tenue muestra, por ejemplo, “1 mod active · first-mod”).

SECCIÓN 03
CÓMO INSTALARLOS

Un mod se instala como cualquier plugin, desde un marketplace.

INSTALAR

```
/plugin install token-chart@your-org
claude plugin install token-chart@your-org
```

El nombre del plugin, una arroba y el nombre del marketplace (ahí `token-chart` y `your-org` son de ejemplo de la documentación).

La primera forma es dentro de una sesión; la segunda, desde tu shell. Si instalás desde el shell con una sesión abierta, corré `/reload-plugins` para cargarlo.

Probar los ejemplos de Anthropic. Anthropic comparte tres mods de muestra en el repositorio **claude-code-playground** (carpeta `claude-code/mods`), sin soporte: cada uno es un plugin completo. Para probar uno, clonás el repo y lo cargás una sola sesión con `--plugin-dir`. Para dejarlo fijo, agregás esa carpeta como marketplace e instalás desde ahí (si movés o borrás el clon, el mod deja de cargar).

CARGAR UN MOD SOLO POR UNA SESIÓN

```
claude --plugin-dir ./ruta/al/mod
```

Apagarlos. Un solo mod: desactivalo o desinstalalo en la pestaña **Installed** de `/plugin`. Todos en una sesión: `claude --safe-mode`. Todos los instalados en todas las sesiones: `"disableAllHooks": true` en `~/.claude/settings.json` (también detiene tus hooks de settings y tu status line).

Antes de instalar uno ajeno. Corré `claude plugin validate ./ese-mod` sobre su carpeta: las líneas `hooks:` y `calls:` listan qué eventos maneja y qué le pide a Claude Code (leer archivos, hacer peticiones de red, iniciar procesos, usar tu plan para llamar a un modelo).

SECCIÓN 04
CÓMO CREARLO

Pedírselo a Claude, o escribirlo vos con tres archivos.

Con Claude. En una sesión interactiva describís lo que querés (por ejemplo, “hacé un mod que muestre la rama de git encima del prompt”). Claude usa una skill incluida llamada `plugin-authoring` y lo escribe en `~/.claude/dev-mods/` más el ID de la sesión. Al guardar el primer archivo, Claude Code te pregunta si activás la recarga en caliente para esa sesión: con “Enable for this session” el mod carga al terminar el turno y recarga en cada turno que lo cambie. Ese mod solo vive en esa sesión: si querés conservarlo, copiá su carpeta a un lugar tuyo y cargala con `--plugin-dir`.

A mano. No hace falta Node, bundler ni paso de compilación: Claude Code carga archivos `.js` y `.ts` directamente.

ARCHIVO	PARA QUÉ SIRVE
`.claude-plugin/plugin.json`	El manifiesto del plugin (nombre, versión, descripción, autor).
`hooks/hooks.json`	Su clave `"modules"` apunta a tu código, por ejemplo `["./register.js"]`; tenerla es lo que convierte el plugin en mod.
`hooks/register.js`	Exporta `register(on)`; cada `on('evento', async ($, e, next) => …)` registra un hook. `$` es la API de mods, `e` el evento y `next` pasa al siguiente.

Ciclo de trabajo. `claude --plugin-dir ./first-mod` recarga el mod al guardar (el estado de variables del módulo se reinicia; lo persistente va en `$.state`). `claude plugin validate` revisa manifiesto y código. `claude plugin test` corre pruebas automáticas sin sesión. Claude Code escribe los tipos de tu versión en `.claude-plugin/types/`; si algo difiere de la documentación, mandan esos tipos. Para compartir: enviá la carpeta o un .zip, ponelo en un marketplace propio (por ejemplo un repo) o enviálo al directorio de Anthropic. Ojo: el nombre no puede parecerse a uno de Anthropic (por ejemplo, empezar con `claude-`).

YA INCLUIDOS EN CLAUDE CODE

Algunas funciones de Claude Code ya son mods.

`/plugin` → pestaña **Installed** los lista bajo **Built-in**: `cc-plugin-diff` (el panel de `/diff`), `cc-plugin-agents-md` (carga `AGENTS.md` como instrucciones del proyecto), `cc-plugin-you-should-know` (una nota encima del prompt cuando un agente lateral ve algo que quizá se te pasó; está apagado hasta que lo activés) y otros de seguridad y telemetría. Su código de `diff`, `agents-md`, `sec-default` y `telemetry` es público en la carpeta `mods` del repositorio de Claude Code, útil como ejemplo.

SECCIÓN 05
USOS

Lo que muestra la documentación oficial, para ver y controlar la sesión.

USO	QUÉ HACE
Contador en vivo	El mod de ejemplo de la guía de creación cuenta las llamadas a herramientas y las muestra junto al spinner mientras Claude trabaja (“Thinking · tool calls: 3…”). Además suma un comando `/tally` que imprime el total.
Pronóstico del contexto	**token-weather** (ejemplo de Anthropic): dibuja encima del prompt un pronóstico de cuánta ventana de contexto te queda.
Freno a comandos riesgosos	**blast-radius** (ejemplo de Anthropic): retiene un comando de shell riesgoso, como `rm -rf` o un force push, muestra qué cambiaría y da botones para seguir o cancelar.
Repetición de ediciones	**replay-theater** (ejemplo de Anthropic): un comando `/replay` que recorre paso a paso las ediciones de archivos que Claude hizo en el último turno.
Reglas sobre herramientas	Un hook en `tool.call` puede rechazar un comando (la documentación muestra uno que bloquea `git push --force`), cambiar sus argumentos o contestar sin ejecutarlo. También puede pausar la llamada y preguntarte con `$.ui.ask`.
Avisos en el transcript	Con `$.ui.log` el mod agrega una línea tenue que Claude no lee, por ejemplo cuando se cambia un archivo `.mdx`. También hay barra de estado (`$.ui.status`) y avisos emergentes (`$.ui.toast`).
Contexto extra al prompt	Un hook en `prompt.submit` puede agregar texto que solo lee Claude: el ejemplo oficial suma la rama actual de git cuando el prompt menciona un PR.
Seguir un turno	Los eventos `turn.start`, `turn.step` y `turn.complete` permiten ver cada pedido al modelo y su uso de tokens, incluida la lectura del caché de prompts.
Tareas con timer y mensajes entre sesiones	La API incluye `$.clock` (timers), `$.process`, `$.http`, `$.model` (llamar a un modelo, que se cobra a tu plan o API key) y `$.session` (leer mensajes y uso, y enviar mensajes a otra sesión o subagente).

SECCIÓN 06
SEGURIDAD Y CONTROL

Un mod es código que corre con tus permisos: tratalo así.

TEMA	QUÉ DICE LA DOCUMENTACIÓN	QUÉ HACER
Alcance	Puede leer y escribir archivos donde tu usuario pueda, iniciar programas, hacer peticiones de red, leer variables de entorno y archivos de settings (con tus claves), ver cada prompt y cada llamada a herramienta, reescribirlos, enviar mensajes a otras sesiones y gastar tu plan o API key.	Instalá solo de autores y marketplaces en los que confiás.
Sin sandbox	Los mods no están aislados. Si activás sandboxing, este aísla los comandos Bash de Claude; un proceso que inicia un mod corre fuera de él.	Leé el código y corré `claude plugin validate` antes de cargarlo.
Aprobaciones	Un mod puede aprobar una llamada antes de que te pregunten. Con settings gestionados o una cuenta Team/Enterprise, por defecto no pasa por encima de tus reglas `ask` ni de hooks `PreToolUse`; sin eso, sí puede. No puede cambiar el aviso de permisos.	Para bloqueos duros, usá reglas de permisos, no un mod.
Fallos	Si un hook falla antes de llamar a `next`, Claude Code lo salta; un hook de bloqueo tiene que cerrar “por fallo” con un `.catch` que niegue la llamada. Cada hook tiene un límite de 10 s de tiempo propio.	No dependas de un mod como única barrera.
Organizaciones	Los administradores controlan los mods con settings gestionados. La opción `allowManagedModsOnly`, bajo `pluginConfigs` de `cc-plugin-sec-default@builtin`, impide cargar mods traídos por los usuarios. También existen `disableAllHooks`, `allowManagedHooksOnly` y `disableSideloadFlags`, y listas `prependPlugins` / `appendPlugins` para fijar el orden de los mods de la organización.	Si tu cliente usa Claude Code en equipo, preguntá cuál es su política.

CÓMO LO USO YO

Uso los mods de Claude Code para ver en tiempo real lo que va pasando en cada sesión: si algo está ocurriendo en esa sesión, lo puedo ver en vivo.

SIGUIENTE PASO

Verificá tu versión con `claude --version` (2.1.287 o más nueva), cloná el repositorio claude-code-playground, cargá **token-weather** con `--plugin-dir` y comprobá con `/plugin` que apareció activo. Después pedile a Claude un mod simple de tu propia sesión, como un contador junto al spinner.

IMPORTANTE: LA INFORMACIÓN PUEDE VARIAR

Todo lo de esta guía (versiones mínimas, dónde se dibuja, comandos, ejemplos, opciones de administración) se revisó el **1 de noviembre de 2026** en la documentación oficial de Claude Code. Los mods son nuevos, la propia documentación avisa que los eventos y métodos pueden cambiar entre versiones, y la disponibilidad de funciones puede variar según tu plan, tu región y tu proveedor de modelos. Confirmá siempre en la documentación vigente y en los tipos que tu versión genera antes de instalar o crear algo.

Basada en la documentación oficial de Claude Code (code.claude.com/docs): vista general de mods, crear un mod, reaccionar a eventos, dibujar en la interfaz, administración para organizaciones, y el repositorio anthropics/claude-code-playground. Preparada por Neiro Agency.